

Intelligenza Artificiale e Laboratorio

Planning, Sistemi a Regole e Incertezza

Roberto Micalizio

a.a. 2025/2026

Università degli Studi di Torino,
Dipartimento di Informatica



Pianificazione nello Spazio dei Piani

- Formulare un problema di pianificazione come un problema di soddisfacimento di vincoli (CSP)
- Usare un CSP consente di ritardare alcune decisioni fino a quando non è strettamente necessario

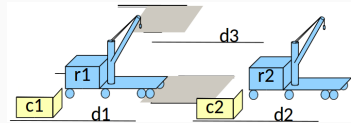
- Formulare un problema di pianificazione come un problema di soddisfacimento di vincoli (CSP)
- Usare un CSP consente di ritardare alcune decisioni fino a quando non è strettamente necessario
- E' utile modificare il linguaggio di rappresentazione del dominio:

FOL $\Rightarrow$ Variabili di Stato

State-Variable Representation

Come costruire un modello espresso da variabili di stato?

- Dato un ambiente E che vogliamo modellare
- B è l'insieme degli oggetti *rilevanti* per il problema che si vuole risolvere
- Esempio:
 - $B = Robots \cup Containers \cup Locs \cup \{nil\}$
 - $Robots = \{r1, r2\}$
 - $Containers = \{c1, c2\}$
 - $Locs = \{d1, d2, d3\}$
- B deve astrarre i dettagli insignificanti (es. colore dei robot, dimensioni, ecc, vanno rappresentati solo se indispensabili per la soluzione del problema)



Proprietà degli oggetti

- Due tipi di proprietà: *rigid* e *varying*
- **rigid:**
 - **proprietà invarianti:** persistono in ogni possibile stato del sistema
Es. la relazione di adiacenza delle posizioni è data dalle coppie
 $adjacent = \{(d1, d2), (d2, d1), (d1, d3), (d3, d1)\}$

Proprietà degli oggetti

- Due tipi di proprietà: *rigid* e *varying*
- **rigid**:
 - **proprietà invarianti**: persistono in ogni possibile stato del sistema
Es. la relazione di adiacenza delle posizioni è data dalle coppie
 $adjacent = \{(d1, d2), (d2, d1), (d1, d3), (d3, d1)\}$
 - nella rappresentazione classica queste proprietà sarebbero modellate come **atomi ground** (relazioni invarianti):
 $\{adjacent(d1, d2), adjacent(d2, d1), adjacent(d1, d3), adjacent(d3, d1)\}$

Proprietà degli oggetti

- Due tipi di proprietà: *rigid* e *varying*
- **rigid**:
 - **proprietà invarianti**: persistono in ogni possibile stato del sistema
Es. la relazione di adiacenza delle posizioni è data dalle coppie
 $adjacent = \{(d1, d2), (d2, d1), (d1, d3), (d3, d1)\}$
 - nella rappresentazione classica queste proprietà sarebbero modellate come **atomi ground** (relazioni invarianti):
 $\{adjacent(d1, d2), adjacent(d2, d1), adjacent(d1, d3), adjacent(d3, d1)\}$
 - In una rappresentazione *state-variable* questi atomi ground corrispondono a **costanti** booleane
 $adjacent(d1, d2) \equiv true$ ($\equiv true$ è omissso in genere)
sono quindi modellate allo stesso modo

- **varying:**
 - possono cambiare nelle transizioni di stato
 - nella rappresentazione classica sono le **relazioni fluenti** che vengono aggiunte/tolte (add/delete list) per effetto delle azioni
Es. $loc(r1, d1)$ è un fluente riguardo la posizione di $r1$

- **varying:**

- possono cambiare nelle transizioni di stato
- nella rappresentazione classica sono le **relazioni fluenti** che vengono aggiunte/tolte (add/delete list) per effetto delle azioni
Es. $loc(r1, d1)$ è un fluente riguardo la posizione di $r1$
- sono **modellate come variabili** a cui possiamo assegnare un valore
- Es. $loc(r1) = d1$
- Uno stato è quindi rappresentato su un insieme *prefissato di variabili*
 $X = \{loc(r1), loc(r2), loc(c1), loc(c2), cargo(r1), cargo(r2)\}$
- ogni variabile di stato ha un **dominio**:
 - $Range(loc(r1)) = Range(loc(r2)) = Locs$
 - $Range(loc(c1)) = Range(loc(c2)) = Robots \cup Locs$
 - $Range(cargo(r1)) = Range(cargo(r2)) = Containers \cup \{nil\}$

- **varying:**

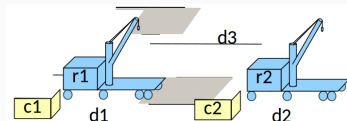
- possono cambiare nelle transizioni di stato
- nella rappresentazione classica sono le **relazioni fluenti** che vengono aggiunte/tolte (add/delete list) per effetto delle azioni
Es. $loc(r1, d1)$ è un fluente riguardo la posizione di $r1$
- sono **modellate come variabili** a cui possiamo assegnare un valore
- Es. $loc(r1) = d1$
- Uno stato è quindi rappresentato su un insieme *prefissato di variabili*
 $X = \{loc(r1), loc(r2), loc(c1), loc(c2), cargo(r1), cargo(r2)\}$
- ogni variabile di stato ha un **dominio**:
 - $Range(loc(r1)) = Range(loc(r2)) = Locs$
 - $Range(loc(c1)) = Range(loc(c2)) = Robots \cup Locs$
 - $Range(cargo(r1)) = Range(cargo(r2)) = Containers \cup \{nil\}$
- (in un linguaggio ad oggetti scriveremmo $r1.loc$)

Come diventa lo stato

- Uno **stato** s_i è una **funzione** che mappa ogni variabile $x \in X$ ad un valore in $Range(x)$

$s1(loc(r1)) = d1, s1(loc(r2)) = d2,$

- $s1(cargo(r1)) = nil, s1(cargo(r2)) = nil,$
 $s1(loc(c1)) = d1, s1(loc(c2)) = d2$



- In altri termini, lo **stato** è un insieme di coppie (x, w) dove per ogni variabile $x \in X$, il valore $w \in Range(x)$ è il valore assegnato alla variabile; es.:

$s1 = \{(loc(r1), d1), (cargo(r1), nil), (loc(c1), d1), \dots\}$

- nella rappresentazione classica avremmo che $s1$ è insieme di *atomi ground positivi*:
 $\{loc(r1, d1), \text{cargo}(r1, nil), loc(c1, d1), loc(r2, d2), \text{cargo}(r2, nil), loc(c2, d2)\}$

Come definire Action Templates

- **Precondizioni:**

Relazioni Costanti:

$rel(t_1, \dots, t_k)$

$\neg rel(t_1, \dots, t_k)$

Variabili di stato:

$sv(t_1, \dots, t_k) = t_0$

$sv(t_1, \dots, t_k) \neq t_0$

dove t_i è un parametro che unifica con un oggetto del dominio

- **Effetti:**

$sv(t_1, \dots, t_k) \leftarrow t_0$

- **Esempi:**

move (r, l, m)	take (r, l, c)
pre : $loc(r)=l, adjacent(l, m)$	pre : $cargo(r)=nil, loc(r)=l, loc(c)=l$
eff : $loc(r) \leftarrow m$	eff : $cargo(r) \leftarrow c,$ $loc(c) \leftarrow r$

- **Applicabilità**

- un'azione a è applicabile in s se:
 - per ogni costante positiva $I \in pre(a)$, $s \models I$ (I è true in s)
 - per ogni costante negativa $I \in pre(a)$, $s \not\models I$ (I è false in s)
 - per ogni test di uguaglianza $sv(t_1, \dots, t_k) = t_0$ in $pre(a)$, allora $sv(t_1, \dots, t_k) = t_0$ vale in s
 - per ogni test di disuguaglianza $sv(t_1, \dots, t_k) \neq t_0$ in $pre(a)$, allora $sv(t_1, \dots, t_k) \neq t_0$ vale in s

- **Transizione di stato**

- se a è applicabile in s , allora $\gamma(s, a) = s'$ t.c.
 $\{(x, w) \mid x \leftarrow w \in eff(a)\} \cup \{(x, w) \in s \mid x \text{ non occorre in } eff(a)\}$
dove (x, w) è la coppia (variabile, valore assegnato alla variabile)

FOL vs. State-Variable Representation

- FOL (o classical) ha origini storiche: planning visto come meccanismo di theorem proving
- FOL e State-Variable sono **equivalenti**:
 - stesso potere espressivo
 - facilmente riconducibili uno all'altro:
 - da predicato a variabile: $P(b_1, \dots, b_k) \rightsquigarrow x_P(b_1, \dots, b_k) = true$
 - da variabile a predicato: $x(b_1, \dots, b_n) = b_0 \rightsquigarrow P_x(b_1, \dots, b_n, b_0)$
 - State-variable ha il vantaggio di essere più **sintetica** e **uniforme** (ogni stato è definito sempre sullo stesso insieme di variabili)
 - State-variable meglio si presta ad **estensioni verso il non classical planning** (es. parziale osservabilità) e all'uso di euristiche domain independent (funzioni di costo, fluenti numerici)

Principio Least Commitment

- Fare scelte solo quando queste sono indispensabili per risolvere una parte del problema
- **Durante la ricerca non porre più vincoli dello stretto necessario**
- Decisioni che è possibile ritardare:
 - **Ordinamenti**: non ordinare le azioni a meno che queste non sia strettamente necessario
 - **Bindings**: non vincolare le variabili a meno che non sia indispensabile unificarle con costanti al fine di conseguire i goal

Least-Commitment Planning: idea

- Ricerca nello spazio dei piani (Plan-Space Planning - PSP)
- **Ogni nodo della ricerca è un piano parzialmente ordinato** con **flaws** (“difetti”)
- Ad ogni passo: rimuovi un difetto per mezzo di **raffinamenti incrementali** del piano parziale corrente
- Se l’algoritmo termina con successo, il piano risultante è completamente istanziato ma **solo parzialmente ordinato**

Least-Commitment Planning: Formalizzazione

Spazio di ricerca = insieme dei **piani parziali**

Un piano è una tupla $\langle A, O, B, L \rangle$ dove

- A insieme di azioni anche solo parzialmente istanziate
 - a_0 (o anche a_{start}) azione senza preconditione e tale che $effects^+(a_0) \equiv s_0$
 - a_∞ (o anche a_{finish}) azione senza effetti e tale che $pre(a_\infty) \equiv s_{goal}$
- O insieme di **vincoli di ordinamento** della forma $(a_i < a_j)$, è una relazione d'ordine solo *parzialmente definita*
- B insieme di **"bindings"** (vincoli) della forma:
 - $v_i = C$:: a v_i è assegnata la costante C
 - $v_i \neq C$:: v_i può assumere qualsiasi valore tranne C
 - $v_i = v_j$:: v_i e v_j assumono lo stesso valore
 - $v_i \neq v_j$:: v_i e v_j devono essere distinte
- L insieme di **causal links** della forma $a_i \xrightarrow{c} a_j$ ad indicare che c è un effetto dell'azione a_i che è necessario (cioè preconditione) per l'azione a_j .
 c è un assegnamento di un valore ad una variabile

Least-Commitment Planning: Formalizzazione

Spazio di ricerca = insieme dei **piani parziali**

Un piano è una tupla $\langle A, O, B, L \rangle$ dove

- A insieme di azioni anche solo parzialmente istanziate
 - a_0 (o anche a_{start}) azione senza preconditione e tale che $effects^+(a_0) \equiv s_0$
 - a_∞ (o anche a_{finish}) azione senza effetti e tale che $pre(a_\infty) \equiv s_{goal}$
- O insieme di **vincoli di ordinamento** della forma $(a_i < a_j)$, è una relazione d'ordine solo *parzialmente definita*
- B insieme di **"bindings"** (vincoli) della forma:
 - $v_i = C$:: a v_i è assegnata la costante C
 - $v_i \neq C$:: v_i può assumere qualsiasi valore tranne C
 - $v_i = v_j$:: v_i e v_j assumono lo stesso valore
 - $v_i \neq v_j$:: v_i e v_j devono essere distinte
- L insieme di **causal links** della forma $a_i \xrightarrow{c} a_j$ ad indicare che c è un effetto dell'azione a_i che è necessario (cioè preconditione) per l'azione a_j .
 c è un assegnamento di un valore ad una variabile

Least-Commitment Planning: Formalizzazione

Spazio di ricerca = insieme dei **piani parziali**

Un piano è una tupla $\langle A, O, B, L \rangle$ dove

- A insieme di azioni anche solo parzialmente istanziate
 - a_0 (o anche a_{start}) azione senza preconditione e tale che $effects^+(a_0) \equiv s_0$
 - a_∞ (o anche a_{finish}) azione senza effetti e tale che $pre(a_\infty) \equiv s_{goal}$
- O insieme di **vincoli di ordinamento** della forma $(a_i < a_j)$, è una relazione d'ordine solo *parzialmente definita*
- B insieme di **"bindings"** (vincoli) della forma:
 - $v_i = C$:: a v_i è assegnata la costante C
 - $v_i \neq C$:: v_i può assumere qualsiasi valore tranne C
 - $v_i = v_j$:: v_i e v_j assumono lo stesso valore
 - $v_i \neq v_j$:: v_i e v_j devono essere distinte
- L insieme di **causal links** della forma $a_i \xrightarrow{c} a_j$ ad indicare che c è un effetto dell'azione a_i che è necessario (cioè preconditione) per l'azione a_j .
 c è un assegnamento di un valore ad una variabile

Least-Commitment Planning: Formalizzazione

Spazio di ricerca = insieme dei **piani parziali**

Un piano è una tupla $\langle A, O, B, L \rangle$ dove

- A insieme di azioni anche solo parzialmente istanziate
 - a_0 (o anche a_{start}) azione senza preconditione e tale che $effects^+(a_0) \equiv s_0$
 - a_∞ (o anche a_{finish}) azione senza effetti e tale che $pre(a_\infty) \equiv s_{goal}$
- O insieme di **vincoli di ordinamento** della forma $(a_i < a_j)$, è una relazione d'ordine solo *parzialmente definita*
- B insieme di **"bindings"** (vincoli) della forma:
 - $v_i = C$: a v_i è assegnata la costante C
 - $v_i \neq C$: v_i può assumere qualsiasi valore tranne C
 - $v_i = v_j$: v_i e v_j assumono lo stesso valore
 - $v_i \neq v_j$: v_i e v_j devono essere distinte
- L insieme di **causal links** della forma $a_i \xrightarrow{c} a_j$ ad indicare che c è un effetto dell'azione a_i che è necessario (cioè preconditione) per l'azione a_j .
 c è un assegnamento di un valore ad una variabile

Least-Commitment Planning: Formalizzazione

Spazio di ricerca = insieme dei **piani parziali**

Un piano è una tupla $\langle A, O, B, L \rangle$ dove

- A insieme di azioni anche solo parzialmente istanziate
 - a_0 (o anche a_{start}) azione senza preconditione e tale che $effects^+(a_0) \equiv s_0$
 - a_∞ (o anche a_{finish}) azione senza effetti e tale che $pre(a_\infty) \equiv s_{goal}$
- O insieme di **vincoli di ordinamento** della forma $(a_i < a_j)$, è una relazione d'ordine solo *parzialmente definita*
- B insieme di **"bindings"** (vincoli) della forma:
 - $v_i = C :: a$ v_i è assegnata la costante C
 - $v_i \neq C ::$ v_i può assumere qualsiasi valore tranne C
 - $v_i = v_j ::$ v_i e v_j assumono lo stesso valore
 - $v_i \neq v_j ::$ v_i e v_j devono essere distinte
- L insieme di **causal links** della forma $a_i \xrightarrow{c} a_j$ ad indicare che c è un effetto dell'azione a_i che è necessario (cioè preconditione) per l'azione a_j .
 c è un assegnamento di un valore ad una variabile

Least-Commitment Planning: Formalizzazione

Spazio di ricerca = insieme dei **piani parziali**

Un piano è una tupla $\langle A, O, B, L \rangle$ dove

- A insieme di azioni anche solo parzialmente istanziate
 - a_0 (o anche a_{start}) azione senza preconditione e tale che $effects^+(a_0) \equiv s_0$
 - a_∞ (o anche a_{finish}) azione senza effetti e tale che $pre(a_\infty) \equiv s_{goal}$
- O insieme di **vincoli di ordinamento** della forma $(a_i < a_j)$, è una relazione d'ordine solo *parzialmente definita*
- B insieme di **"bindings"** (vincoli) della forma:
 - $v_i = C :: a$ v_i è assegnata la costante C
 - $v_i \neq C :: a$ v_i può assumere qualsiasi valore tranne C
 - $v_i = v_j :: a$ v_i e v_j assumono lo stesso valore
 - $v_i \neq v_j :: a$ v_i e v_j devono essere distinte
- L insieme di **causal links** della forma $a_i \xrightarrow{c} a_j$ ad indicare che c è un effetto dell'azione a_i che è necessario (cioè preconditione) per l'azione a_j .
 c è un assegnamento di un valore ad una variabile

Least-Commitment Planning: Formalizzazione

Spazio di ricerca = insieme dei **piani parziali**

Un piano è una tupla $\langle A, O, B, L \rangle$ dove

- A insieme di azioni anche solo parzialmente istanziate
 - a_0 (o anche a_{start}) azione senza preconditione e tale che $effects^+(a_0) \equiv s_0$
 - a_∞ (o anche a_{finish}) azione senza effetti e tale che $pre(a_\infty) \equiv s_{goal}$
- O insieme di **vincoli di ordinamento** della forma $(a_i < a_j)$, è una relazione d'ordine solo *parzialmente definita*
- B insieme di **"bindings"** (vincoli) della forma:
 - $v_i = C :: a$ v_i è assegnata la costante C
 - $v_i \neq C :: v_i$ può assumere qualsiasi valore tranne C
 - $v_i = v_j :: v_i$ e v_j assumono lo stesso valore
 - $v_i \neq v_j :: v_i$ e v_j devono essere distinte
- L insieme di **causal links** della forma $a_i \xrightarrow{c} a_j$ ad indicare che c è un effetto dell'azione a_i che è necessario (cioè preconditione) per l'azione a_j .
 c è un assegnamento di un valore ad una variabile

Sia π il piano del nodo corrente:

- Una preconditione p per un'azione b è un **open goal** se non c'è un link causale a supporto di p , cioè manca ? $\xrightarrow{P} b$ in L
- **Risolvere il difetto aggiungendo il link causale mancante:**
 1. trova un'azione a (possibilmente già in π , o in alternativa da aggiungere a π) tale che:
 - p può appartenere a $\text{effect}^+(a)$ (previa unificazione)
 - a può precedere b in π
 2. istanzia l'azione a in modo che asserisca p
(istanziamento least-commitment $\rightarrow$ solo lo stretto necessario)
 3. aggiungi un vincolo di precedenza $a \prec b$ in O
 4. aggiungi un causal link $a \xrightarrow{P} b$ in L

Sia π il piano del nodo corrente:

- Una preconditione p per un'azione b è un **open goal** se non c'è un link causale a supporto di p , cioè manca ? $\xrightarrow{p} b$ in L
- **Risolvere il difetto aggiungendo il link causale mancante:**
 1. trova un'azione a (possibilmente già in π , o in alternativa da aggiungere a π) tale che:
 - p può appartenere a $effect^+(a)$ (previa unificazione)
 - a può precedere b in π
 2. istanzia l'azione a in modo che asserisca p
(istanziamento least-commitment $\rightarrow$ solo lo stretto necessario)
 3. aggiungi un vincolo di precedenza $a \prec b$ in O
 4. aggiungi un causal link $a \xrightarrow{p} b$ in L

Flaws: Open Goals

Sia π il piano del nodo corrente:

- Una preconditione p per un'azione b è un **open goal** se non c'è un link causale a supporto di p , cioè manca ? $\xrightarrow{p} b$ in L
- **Risolvere il difetto aggiungendo il link causale mancante:**
 1. trova un'azione a (possibilmente già in π , o in alternativa da aggiungere a π) tale che:
 - p può appartenere a $effect^+(a)$ (previa unificazione)
 - a può precedere b in π
 2. istanzia l'azione a in modo che asserisca p
(istanziamento least-commitment -> solo lo stretto necessario)
 3. aggiungi un vincolo di precedenza $a \prec b$ in O
 4. aggiungi un causal link $a \xrightarrow{p} b$ in L

Flaws: Open Goals

Sia π il piano del nodo corrente:

- Una preconditione p per un'azione b è un **open goal** se non c'è un link causale a supporto di p , cioè manca ? $\xrightarrow{p} b$ in L
- **Risolvere il difetto aggiungendo il link causale mancante:**
 1. trova un'azione a (possibilmente già in π , o in alternativa da aggiungere a π) tale che:
 - p può appartenere a $effect^+(a)$ (previa unificazione)
 - a può precedere b in π
 2. istanzia l'azione a in modo che asserisca p
(istanziamento least-commitment $\rightarrow$ solo lo stretto necessario)
 3. aggiungi un vincolo di precedenza $a \prec b$ in O
 4. aggiungi un causal link $a \xrightarrow{p} b$ in L

Sia π il piano del nodo corrente:

- Una preconditione p per un'azione b è un **open goal** se non c'è un link causale a supporto di p , cioè manca ? $\xrightarrow{p} b$ in L
- **Risolvere il difetto aggiungendo il link causale mancante:**
 1. trova un'azione a (possibilmente già in π , o in alternativa da aggiungere a π) tale che:
 - p può appartenere a $effect^+(a)$ (previa unificazione)
 - a può precedere b in π
 2. istanzia l'azione a in modo che asserisca p
(istanziamento least-commitment $\rightarrow$ solo lo stretto necessario)
 3. aggiungi un vincolo di precedenza $a \prec b$ in O
 4. aggiungi un causal link $a \xrightarrow{p} b$ in L

Sia π il piano del nodo corrente:

- Una preconditione p per un'azione b è un **open goal** se non c'è un link causale a supporto di p , cioè manca ? $\xrightarrow{p} b$ in L
- **Risolvere il difetto aggiungendo il link causale mancante:**
 1. trova un'azione a (possibilmente già in π , o in alternativa da aggiungere a π) tale che:
 - p può appartenere a $effect^+(a)$ (previa unificazione)
 - a può precedere b in π
 2. istanzia l'azione a in modo che asserisca p
(istanziamento least-commitment $\rightarrow$ solo lo stretto necessario)
 3. aggiungi un vincolo di precedenza $a \prec b$ in O
 4. aggiungi un causal link $a \xrightarrow{p} b$ in L

Flaws: Open Goals - Example

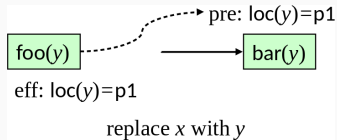
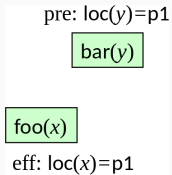
pre: $\text{loc}(y)=p1$

$\text{bar}(y)$

$\text{foo}(x)$

eff: $\text{loc}(x)=p1$

Flaws: Open Goals - Example



Sia π il piano del nodo corrente:

- Dato un link causale $l : a \xrightarrow{p} b$,
- un'azione c minaccia (*threatens*) il link l se:
 - c può modificare il valore di verità di p e può posizionarsi tra a e b ;
oppure
 - c può produrre p : il link l impone che sia a a produrre p per b e non un'altra azione c
se anche c può produrre p per b questa è un'opportunità che deve essere investigata in un altro ramo dello spazio di ricerca
- c viene anche detta **clobber**

- Situazione:

- $l : a \xrightarrow{p} b$, link causale in L
- c azione in A clobber per l

- Risolvere le minacce:

- **Promotion**: imporre il vincolo $c \prec a$
- **Demotion**: imporre il vincolo $b \prec c$
- **Separation**: imporre un vincolo di *non-codesignation* in modo tale che l'effetto di c non unifichi con p

Es. $p : pos(Block1) = Table$, l'effetto di c potrebbe essere $pos(x) = Hand$, allora il vincolo $x \neq Block1$ deve essere aggiunto a B

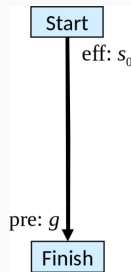
Algoritmo Plan-Space Planning (PSP)

```
PSP( $\Sigma$ ,  $\pi$ )  
   $Flaws(\pi) \leftarrow OpenGoals(\pi) \cup Threats(\pi)$   
  if ( $Flaws(\pi) = \emptyset$ ) then return  $\pi$   
  arbitrarily select  $f \in Flaws(\pi)$   
   $R \leftarrow \{all\ feasible\ resolvers\ for\ f\}$   
  if  $R = \emptyset$  then return failure  
  nondeterministically select  $\rho \in R$   
   $\pi' \leftarrow \rho(\pi)$   
  return PSP( $\Sigma, \pi'$ )
```

¹ $\pi = \langle A, O, B, L \rangle$

Algoritmo Plan-Space Planning (PSP)

```
PSP( $\Sigma$ ,  $\pi$ )  
   $Flaws(\pi) \leftarrow OpenGoals(\pi) \cup Threats(\pi)$   
  if ( $Flaws(\pi) = \emptyset$ ) then return  $\pi$   
  arbitrarily select  $f \in Flaws(\pi)$   
   $R \leftarrow \{all\ feasible\ resolvers\ for\ f\}$   
  if  $R = \emptyset$  then return failure  
  nondeterministically select  $\rho \in R$   
   $\pi' \leftarrow \rho(\pi)$   
  return PSP( $\Sigma, \pi'$ )
```



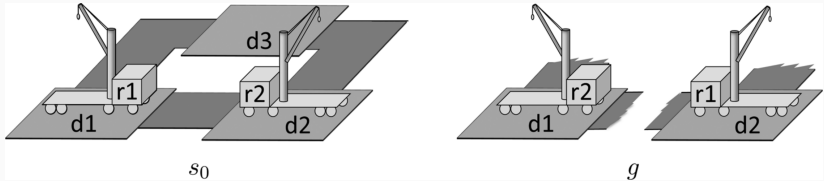
Inizializzazione argomenti:

- Σ è l'STS che modella il dominio
- $\pi^1 = (\{a_0, a_\infty\}, \{a_0 \prec a_\infty\}, \emptyset, \emptyset)$

¹ $\pi = \langle A, O, B, L \rangle$

PSP è corretto e completo

- ritorna un piano parzialmente ordinato π tale che qualsiasi ordinamento totale delle sue azioni raggiunge il goal
- dove il dominio lo consenta, le azioni non strettamente sequenziali possono essere eseguite in parallelo



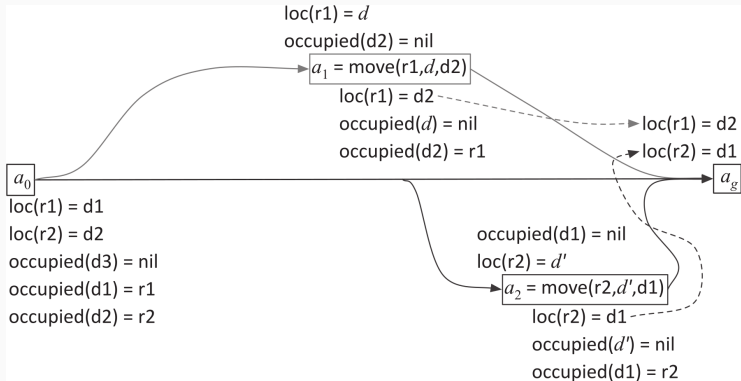
$s_0 = \{\text{loc}(r1) = d1, \text{loc}(r2) = d2, \text{occupied}(d1) = T,$
 $\text{occupied}(d2) = T, \text{occupied}(d3) = F\};$
 $g = \{\text{loc}(r1) = d2, \text{loc}(r2) = d1\}.$

move(r, d, d')
pre: $\text{loc}(r) = d, \text{occupied}(d') = \text{nil}$
eff: $\text{loc}(r) \leftarrow d',$
 $\text{occupied}(d) \leftarrow \text{nil},$
 $\text{occupied}(d') \leftarrow r$

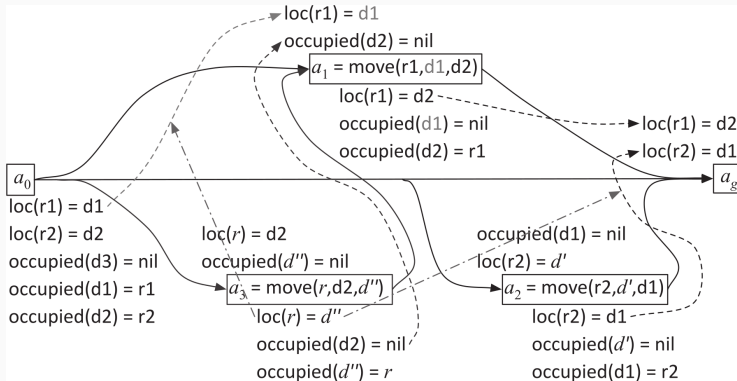


Stato iniziale della ricerca

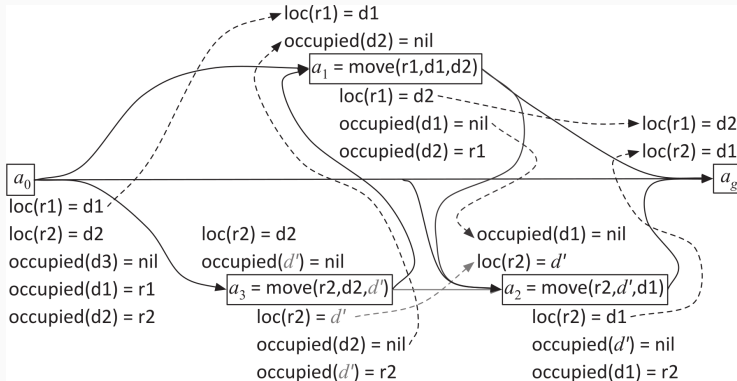
PSP - Esempio



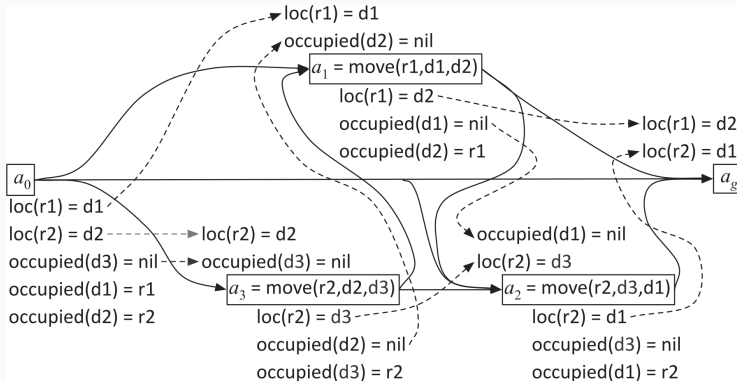
Risolvere gli *open goals* in s_g comporta l'aggiunta di due azioni e rispettivi causal links



Risolve gli *open goals* di a_1 : in che modo?



Risolve gli *open goals* di a_2 : in che modo?



Risolve gli *open goals* di a_3

Least-Commitment Planing: Glossario

Totally ordered plan : esiste un numero sufficiente di relazioni in O tale che ogni azione in A risulta ordinata rispetto ad ogni altra azione

Partially ordered plan : un piano che non è totalmente ordinato, e privo di difetti: ogni sua possibile linearizzazione porta dallo stato iniziale al soddisfacimento del goal

Partial plan : piano parzialmente ordinato che non è ancora una soluzione perché contiene difetti, e le sue azioni possono essere solo parzialmente istanziate

Open goal : per una data azione a , quando almeno un atomo delle precondizioni di a non è fornita nè dallo stato iniziale, nè da un'altra azione del piano corrente.

Threat : una qualsiasi azione c che minaccia un link causale $a \xrightarrow{P} b$ esistente tra le due azioni a e b